

Convolution-Friendly Image Compression with FHE

Axel Mertens¹ , Georgio Nicolas¹ , and Sergi Rovira² 

¹ COSIC, KU Leuven

{axel.mertens, georgio.nicolas}@esat.kuleuven.be

² Technology Innovation Institute (TII)

sergi.rovira@tii.ae

Abstract. During the past few decades, the field of image processing has grown to cradle hundreds of applications, many of which are outsourced to be computed on trusted remote servers. More recently, Fully Homomorphic Encryption (FHE) has grown in parallel as a powerful tool enabling computation on encrypted data, and transitively on untrusted servers. As a result, new FHE-supported applications have emerged, but not all have reached practicality due to hardware, bandwidth or mathematical constraints inherent to FHE. One example is processing encrypted images, where practicality is closely related to bandwidth availability. In this paper, we propose and implement a novel technique for FHE-based image compression and decompression. Our technique is a stepping stone towards practicality of encrypted image-processing and applications such as private inference, object recognition, satellite-image searching or video editing.

Inspired by the JPEG standard, and with new FHE-friendly compression/decompression algorithms, our technique allows a client to compress and encrypt images before sending them to a server, greatly reducing the required bandwidth. The server homomorphically decompresses a ciphertext to obtain an encrypted image to which generic pixel-wise processing or convolutional filters can be applied. To reduce the round-trip bandwidth requirement, we also propose a method for server-side post-processing compression.

Using our pipeline, we demonstrate that a high-definition grayscale image (1024×1024) can be homomorphically decompressed, processed and re-compressed in ~ 8.1 s with a compression ratio of 100/34.4 on a standard personal computer without compromising on fidelity.

Keywords: Secure Image Processing · Fully Homomorphic Encryption · Privacy-Enhancing Technologies

1 Introduction

Data compression is a crucial subfield within the domain of data science, playing a pivotal role in efficiently managing the flood of data generated and processed daily. Its significance is clear in diverse applications such as

internet browsing, telecommunications, and data storage. Recent developments in Fully Homomorphic Encryption (FHE) have opened the door to new privacy-preserving applications which use data compression such as private satellite searching and object recognition. FHE allows us to perform additions and multiplications directly on ciphertexts without the need of the secret key. Unfortunately, using FHE has the inherent drawback of ciphertext expansion - a blow-up in the size of a plaintext when encrypted. Therefore, adding privacy to data compression and processing applications requires a robust data compression algorithm tailored for FHE.

This paper takes a big step in this direction, specifically for the homomorphic compression and processing of images. Possible applications of our work include satellite image-search, private object detection, facial recognition, etc. In this paper, we propose a compression scheme for the encryption of natural images in CKKS [11], that uses a modification of the method used in the JPEG-1 standard [21, 29]. Our pipeline is as in figure 1, where an image is compressed and encrypted before being sent to a server for processing. The server performs decompression, processing and compression all in the encrypted domain, before sending it back to the client for decryption. The suggested scheme is designed to be compatible with server-side image processing with convolutional filters and pixel-wise image processing.

The compression (decompression, resp.) algorithm is associative with encryption (decryption, resp.) algorithm, improving client-side performance.

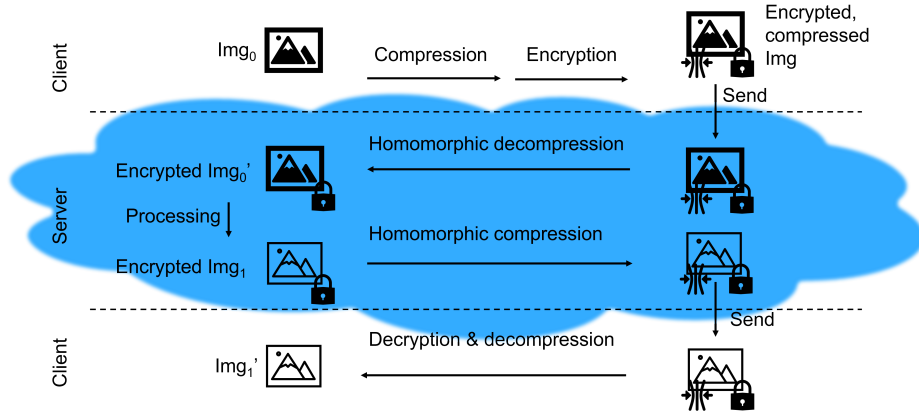


Fig. 1: Sketch of our pipeline

1.1 Our Contributions

Our contributions can be summarized as follows:

1. We build the first end-to-end pipeline for efficient encrypted image compression and decompression based on FHE.
2. We demonstrate how to efficiently homomorphically apply a convolutional filter and pixel by pixel operations to a decompressed encrypted image.
3. We provide extensive reproducible benchmarks based on a proof-of-concept implementation of our framework.

In more detail, starting from the JPEG standard, we replace the entropy encoding with a constant cut-off. Not only is this cheap and efficient to perform, it also proves to be successful in practice. To the best of our knowledge, this technique is novel. We combine this compression scheme with the state-of-the-art FHE scheme CKKS [11], taking full advantage of its ciphertext packing technique and support for approximate arithmetic. We propose a trick that enables convolution-based image processing on images that are packed in this manner, avoiding a costly unpacking step. Together, this results in a homomorphic compression scheme that is fast and effective, while maintaining great image quality ($SSI \geq 0.95$).

Furthermore, our scheme is versatile and modular in the sense that all of its components not only complement each-other, but also can be used independently depending on the user’s requirements. For instance, we can use the compression and decompression components to enable performing other kinds of processing.

It ultimately depends on the application whether the communication channel is the bottleneck, or if the server computation is the most costly. This paper tackles problem with privacy-preserving image processing which exists case when communication is expensive.

1.2 Related Work

Little research can be found on the topic of compression and decompression of encrypted images. There exists more work on the related topic of ciphertext compression, where the ciphertext is compressed in a manner that is agnostic to the structure of underlying plaintext data. Such techniques are independent of the scope of this work, and this could be combined with ours in certain cases. There also exists literature about homomorphic image processing, most notably a demo published by Zama [33] which we compare with in Section 4.4.

In 2017, Canard et al. [9] analyzed the execution of a run-length coding scheme in FHE. The authors came to the conclusion that homomorphic run-length coding is impractical, due to the large number of comparisons it requires. They further argue that lossless compression in FHE cannot be guaranteed, stating that any lossless compression algorithm has a lousy worst-case behavior. In FHE, every algorithm achieves its worst-case behavior. Other attempts at homomorphic compression, such as [17] and [31], are interesting but remain impractical due to this very issue. The authors of [27] attempt to tackle the problem of homomorphic Huffman decoding, but also remain far from practicality and proposing a solution for compression in the encrypted domain. We compare our results with theirs in Section 4.2.

In [25], the authors suggest an FHE-friendly homomorphic compression scheme. Unfortunately, they fall short of describing how to encode a compressed image or perform processing on it. Moreover, the method results in a high loss of image fidelity in comparison.

In [16], the authors suggest a technique using invertible Bloom filters to compress an encrypted sparse array, with a joint decompression - decryption function. This method is more general than the one we propose in this paper and only requires an additive homomorphic scheme. However, we still miss the discussion on encrypted decompression and find that our remains more effective for images. We require fewer operations overall and achieve a higher compression rate. The compression scheme of [34] remains versatile, as each pixel remains stored separately. Where our proposal loses versatility because of our packing method, their system loses in compression ratio.

In [24], the author studies information leakage in systems that combine both compression and encryption. A number of attacks that use only the compression ratio as a side-channel are listed. This study serves as a stern warning that for perfect privacy, a compression ratio should remain secret.

Related but less relevant work discusses homomorphic neural networks: [30] tackles related issues in limiting the memory required during homomorphic evaluation of a residual neural network. [3], [23] and [26] describe methods for optimizing neural networks in FHE. While this is a very interesting and related topic, there remains not much consideration on how to limit the communication cost between client and server.

2 Preliminaries

In this section, we introduce some background on FHE, placing special emphasis on the CKKS scheme. We also provide the necessary background on image compression. We refer the interested reader to the references provided in this section for a more in-depth explanation of these topics.

2.1 Notation

We will denote by $\mathcal{R}$ and $\mathcal{R}_q$ the rings of polynomials $\mathbb{Z}[X]/(X^N + 1)$ and $\mathbb{Z}_q[X]/(X^N + 1)$ respectively, for positive integers q and N .

We use $x \leftarrow D$ to denote sampling x according to a distribution D . If D is finite, we assume the sampling is done uniformly at random.

The complex canonical embedding $\tau : \mathcal{S} \rightarrow \mathbb{C}^N$ is defined as $\tau(a) = (a(\eta^j))_{j \in \mathbb{Z}_M^*}$ for $\eta = \exp(2\pi i/M)$, where $M = 2N$ and $\mathbb{Z}_M^* = \{x \in \mathbb{Z}_M : \gcd(x, M) = 1\}$.

In CKKS, the maps $\tau'_n : \mathcal{S} \rightarrow \mathbb{C}^n$, a variant for $n \leq N/2$ of the canonical embedding τ , are used to encode and decode.

Let L be the maximum number of levels fixed in the CKKS setup and let q_i be the prime modulus for $i \in [L]$. Fix δ and set $\alpha = (L+1)/\delta$. Consider the products

$\tilde{Q}_j = \prod_{i=j\alpha}^{(j+1)\alpha-1} q_i$ for $j \in \{0, \dots, \delta\}$. Given a level ℓ and $\delta' = \lceil (\ell + 1)/\alpha \rceil$, we can decompose $a \in \mathcal{R}_\ell$ as follows:

$$\mathcal{WD}_\ell(a) = \left(\left[a \frac{\tilde{Q}_0}{Q_\ell} \right]_{\tilde{Q}_0}, \dots, \left[a \frac{\tilde{Q}_{\delta'-1}}{Q_\ell} \right]_{\tilde{Q}_{\delta'-1}} \right) \in \mathcal{R}^{\delta'},$$

$$\mathcal{PW}_\ell(a) = \left(\left[a \frac{Q_\ell}{\tilde{Q}_0} \right]_{Q_\ell}, \dots, \left[a \frac{Q_\ell}{\tilde{Q}_{\delta'-1}} \right]_{Q_\ell} \right) \in \mathcal{R}_{Q_\ell}^{\delta'}.$$

For any $(a, b) \in \mathcal{R}_\ell^2$, we have that

$$\langle \mathcal{WD}_\ell(a), \mathcal{PW}_\ell(b) \rangle \equiv a \cdot b \pmod{Q_\ell}.$$

2.2 Background on Fully Homomorphic Encryption

A Fully Homomorphic Encryption (FHE) scheme allows us to perform arbitrary computations over encrypted inputs without decrypting them first. That is, given a ciphertext ct encrypting a plaintext m and an arbitrary function f , we can obtain a new ciphertext ct' encrypting $f(m)$ without decrypting ct .

Since the first FHE scheme [18], research in FHE has seen rapid growth, leading to a wide range of schemes built from different techniques and security assumptions [8, 11, 13, 15].

It is important to remark that FHE schemes can only directly compute additions (XOR) and multiplications (AND) between ciphertexts. More complex functions are represented as arithmetic (boolean) circuits built from these two basic operations. The encryption function of most FHE schemes adds a random element to the message, commonly referred to as *error* or *noise* in the literature. When a multiplication or an addition is performed, this error increases. After a given number of operations, the error in a ciphertext may become too large, overwriting the encapsulated message. This leads to an inability of recovering the original message by the decryption algorithm. To solve this problem, an additional algorithm called *bootstrapping* is added to the scheme. This procedure reduces the noise of a ciphertext, allowing further operations. Unfortunately, bootstrapping is a costly operation. Therefore, it is common practice to set parameters of an FHE scheme in such a way that we can evaluate a function of our choice without the need for a bootstrapping operation. This approach is called *leveled* FHE and is the one that we take in this work.

Definition 1 (Homomorphic Encryption scheme) *A public-key homomorphic encryption scheme $\mathcal{E}$ consists of a set of probabilistic polynomial-time algorithms (KeyGen, Enc, Dec, Eval) such that:*

- **KeyGen**(1^λ): *outputs the secret key sk , the public key pk and the evaluation key evk given the security parameter λ . The evaluation key is also public, and it is used to perform the homomorphic operations over ciphertexts.*
- **Enc** _{pk} (m): *Outputs a ciphertext ct encrypting m under the public key pk .*

- $\text{Dec}_{sk}(ct)$: Outputs a message m . If the algorithm cannot recover m from ct , the output is $\perp$.
- $\text{Eval}_{evk}(f; ct_1, \dots, ct_n)$: Given a function f and a list of ciphertexts $ct_1, \dots, ct_n$, outputs a ciphertext ct_f such that $\text{Dec}_{sk}(ct_f) = f(m_1, \dots, m_n)$, where $ct_i \leftarrow \text{Enc}_{pk}(m_i)$ for all $i \in \{1, \dots, n\}$.

Among the most popular FHE schemes available today [6, 7, 11, 12, 15], we consider CKKS [11] to be the best scheme for encrypted image processing for the following reasons:

- It allows us to compute directly on floating-point numbers, which is crucial for the fast evaluation of our compression and decompression algorithms. Furthermore, this allows us to work with non-integer kernels when performing image processing.
- The other key feature of CKKS is ciphertext packing, which allows us to encrypt multiple blocks of an image inside a single ciphertext. We expand on our use of ciphertext packing in Section 3.3.

We want to point out that very recently, floating-point operations have been added to TFHE [4], another rapidly growing FHE scheme. Nevertheless, CKKS remains the best option for encrypted image processing since TFHE does not support ciphertext packing, which is crucial for our application. Furthermore, the authors of [1] add functional bootstrapping to CKKS. This has direct implications to our framework, meaning that we can now apply a wider range of filters and transformations to the decompressed images on the server side. Unfortunately, we have not been able to test this extension since the technique of [1] has not yet been incorporated into OpenFHE, the FHE library we used to implement our proof of concept.

2.3 Background on CKKS

The CKKS scheme [11] is an approximate homomorphic encryption scheme. The key difference between CKKS and other schemes, such as BGV [7] or TFHE [12], is that the decryption function of CKKS does not remove the encryption noise. This makes the decryption structure to be $m+e \bmod q$ where m is the encrypted message, e is the encryption error and q a suitable modulus. Intuitively, this approach simulates computer numerical errors. That is, if e is small enough compared to m , it only alters the least significant bits of m . Therefore, we can operate on $m + e$ knowing that only some portion of m has been altered and thus contains errors.

Another characteristic of CKKS is the native support for ciphertext packing, which allows for bundling multiple encrypted messages into a single ciphertext. More importantly, this provides the capability of performing computations between ciphertexts in a SIMD manner, reducing both effective memory requirements and computational complexity.

Below is a more rigorous definition of the main algorithms we employ from CKKS. We provide them to make the paper as self-contained as possible, the

reader may skip these details if they are not interested in consulting the code from our proof-of-concept implementation.

- Setup(1^λ). Fix $L \geq 0$ (largest ciphertext modulus level). Given the security parameter λ , output the ring dimension N . Set $\chi_{\text{key}}, \chi_{\text{err}}, \chi_{\text{enc}}$ the secret, error and encryption distributions as follows. For an positive integer h , the secret key distribution χ_{key} follows a uniform distribution over the set of signed binary vectors in $\{0, \pm 1\}^N$ whose Hamming weight (the number of nonzero coefficients) is exactly h . The error distribution χ_{err} chooses a polynomial s by sampling its coefficients independently from the discrete Gaussian distribution of variance σ^2 for $\sigma > 0$. The encryption key distribution χ_{enc} draws each entry in the vector from $\{0, \pm 1\}$, with probability $1/4$ for each of $+1$ and -1 , and probability being zero $1/2$.
- KeyGen. Sample $s \leftarrow \chi_{\text{key}}, a \leftarrow \mathcal{R}_{Q_L}, e \leftarrow \chi_{\text{err}}$. Set $\text{sk} \leftarrow (1, s)$ and $\text{pk} \leftarrow (b, a) \in \mathcal{R}_{Q_L}^2$, where $b = -as + e \pmod{Q_L}$.
- Encode($\mathbf{x}, \Delta$). Given $\mathbf{x} \in \mathbb{C}^n$, output $m = \lfloor \tau_n'^{-1}(\Delta \cdot x) \rfloor \in \mathcal{R}$.
- Decode(m, Δ). Given $m \in \mathcal{R}$, output $\mathbf{x} = \tau_n'(m/\Delta) \in \mathbb{C}^n$.
- Enc_{pk}(m). Given $m \in \mathcal{R}$, sample $v \leftarrow \chi_{\text{enc}}$ and $e_0, e_1 \leftarrow \chi_{\text{err}}$. Output $\text{ct} = v \text{pk} + (m + e_0, e_1) \pmod{Q_L}$.
- Dec_{sk}(ct). Given $\text{ct} = (b, a) \in \mathcal{R}_{Q_\ell}^2$, output $m = b + a \cdot s \pmod{Q_\ell}$.
- Rescale(ct, q_ℓ). Given $\text{ct} \in \mathcal{R}_{Q_\ell}^2$, output $\text{ct}' = \lfloor q_\ell^{-1} \cdot \text{ct} \rfloor \pmod{Q_{\ell-1}}$.
- CAdd($\text{ct}, \mathbf{x}$). Given $\text{ct} = (b, a) \in \mathcal{R}_{Q_\ell}^2$ with scaling factor $\Delta^{\ell'}$ and $\mathbf{x} \in \mathbb{C}^n$, encode $\mathbf{x}$ obtaining $m = \text{Encode}(\mathbf{x}, \Delta^{\ell'})$, and output $\text{ct}_{\text{cadd}} \leftarrow (b + m, a) \pmod{Q_\ell}$.
- Add(ct_1, ct_2). Given $\text{ct}_1, \text{ct}_2 \in \mathcal{R}_{Q_\ell}^2$, output $\text{ct}_{\text{add}} = \text{ct}_1 + \text{ct}_2 \pmod{Q_\ell}$.
- CMult($\text{ct}, \mathbf{x}$). Given $\text{ct} = (c_0, c_1) \in \mathcal{R}_{Q_\ell}^2$ and $\mathbf{x} \in \mathbb{C}^n$, compute $m = \text{Encode}(\mathbf{x}, \Delta)$ and output $\text{ct}_{\text{cmult}} = (c_0 \cdot m, c_1 \cdot m) \pmod{Q_\ell}$.
- KeySwitchGen_{sk}(s'). For auxiliary modulus $P = \prod_{i=0}^k p_i$, sample $a'_{\text{pk}} \leftarrow \mathcal{R}_{PQ_L}$ and $e'_k \leftarrow \chi_{\text{err}}$. For a predefined δ , output

$$\text{swk} = (\text{swk}_0, \text{swk}_1) = (\{b'_k\}_{k=0}^{\delta-1}, \{a'_k\}_{k=0}^{\delta-1}) \in \mathcal{R}_{PQ_L}^{2 \times \delta},$$

where

$$b'_k = -a'_k \cdot s + e'_k + P \cdot \mathcal{PW}'(s')_k \pmod{PQ_L}.$$

- KeySwitch_{swk}(s'). Given $\text{ct} = (c_0, c_1) \in \mathcal{R}_{Q_\ell}^2, \text{swk} = (\text{swk}_0, \text{swk}_1)$, output

$$\left(c_0 + \left\lfloor \frac{\langle \mathcal{WD}_\ell(c_1), \text{swk}_0 \rangle}{P} \right\rfloor, \left\lfloor \frac{\langle \mathcal{WD}_\ell(c_1), \text{swk}_1 \rangle}{P} \right\rfloor \right) \pmod{Q_\ell}$$

- Mult(ct_1, ct_2). Given two ciphertexts ct_1 and ct_2 , compute $\text{ct}'_1 = \text{Rescale}(\text{ct}_1, q_\ell)$ and $\text{ct}'_2 = \text{Rescale}(\text{ct}_2, q_\ell)$. Let $(d_0, d_1, d_2) = (b_1 \cdot b_2, a_1 \cdot b_2 + a_2 \cdot b_1, a_1 \cdot a_2) \pmod{Q_{\ell-1}}$. Output

$$\text{ct}_{\text{mult}} \leftarrow (d_0, d_1) + \text{KeySwitch}_{\text{evk}}(0, d_2) \pmod{Q_{\ell-1}}.$$

2.4 Background on Image Compression

When using lossy compression, data is represented more compactly, with the aim of retaining as much fidelity as possible to the original source before compression. When considering images, there are different methods to measure the level of fidelity or closeness. The measure closest to how the human eye perceives images is the Structural Similarity Index (SSI) [32]:

Definition 2 (Structural Similarity Index (SSI)) *SSI is defined as:*

$$SSI(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$
 where μ_i is the pixel sample mean of image i , σ_i^2 is the variance of image i , σ_{xy}^2 is the covariance of images x and y , $c_1 = (0.01L)^2$, $c_2 = (0.03L)^2$, with L the dynamic range of the pixel values.

It compares an imperfect image to its perfect version, and is a value between -1 and 1, where 1 is a perfect match, -1 is a perfectly negative correlation, and 0 means the two images are very different. The SSI is typically good at detecting structural deformations (stretching, rotation, etc.) and other forms of degradation such as blurriness or blockiness. Images with values of 0.95 are considered to be of very good quality [35].

Another important metric is the compression ratio:

Definition 3 (Compression ratio) *Let B_0 be the number of bytes needed to represent some data X , and let B_1 be the number of bytes needed for the same, compressed data X' . The compression ratio is then $\frac{B_0}{B_1}$, also written as $B_0:B_1$.*

An ideal compression scheme should have a high compression ratio, while maintaining an SSI that is close to 1.

3 Our Technique

Most traditional compression algorithms [19, 20, 28] used in the clear are unfit for FHE. They often contain comparisons and do not have a constant run-time. FHE inflicts its own special set of requirements upon an algorithm. The most important property of a homomorphic compression scheme is that no additional data or metadata about the image may leak, even through side-channels such as the compression ratio or running-time. This is precisely why implementing the JPEG standard as is in FHE would not be possible as we discuss later. Two more critical properties are efficient compression and decompression and that ciphertext packing should be done in a way that a ciphertext structurally reflects the compression of a plaintext. With some inefficient packing methods, the ciphertext length would not be influenced by a (slightly) shorter plaintext. And last but not least, the scheme should allow for both compression and decompression of the message without needing to decrypt it between the operations.

As an extra requirement for image processing, we also target having the encrypted, decompressed form of the message support efficient image processing.

The type of image processing we choose to focus on is using convolutional filters, but we also look at pixel-wise processing methods. Furthermore, our technique can also handle colored images, both in the RGB format as in the YCbCr format, depending on the application. Without discussing this too far, it is clear to see how the proposed technique can be extended towards dealing with multi-channel images. Moreover, with the recent addition of functional bootstrapping to CKKS [1] our framework can be to support more complex image processing techniques.

3.1 Compression Algorithm

In this section we introduce our algorithms for compression and decompression. We want to emphasize that we present them in unencrypted format, as the client will run them. The reader should keep in mind that the server will execute them in the encrypted domain. Specifically, in the server side, the image data ($\mathbf{a}$, $\mathbf{A}$, $\mathbf{B}$) are encrypted, while the sizes of vectors and matrices (m , n , c) remain public. The algorithm we suggest is based on the JPEG-standard. For readability, we list our symbols and notations in table 1.

As shown in Algorithm 2, we split the image into $m \times m$ -sized blocks. We first center every pixel value in the $[-128, 128[$ interval. We call this operation *Level()*, while the inverse operation is called *Unlevel()*. Then, we apply a discrete cosine transform (DCT), which is done via two fixed-point matrix multiplications with DCT matrix T_m . This gives us representations of the frequency spectrum of each block of the image. The next step, still following the JPEG standard, is to apply a quantization, which scales each element of this frequency representation (lines 4-6 of alg. 2). The quantization matrix Q_m is defined through exhaustive testing and standardization. Hence, we are bound to the quantization matrices existing in prior research. We have only found a reliable matrix for $m = 8$ and a slightly less mature matrix for $m = 16$ [2].

Table 1: List of Symbols

Symbol	Meaning
λ	Security Parameter
Q_m	Quantization Matrix
q	Quality Parameter
k	Quantization Matrix Scaling Factor
c, c_i	Cut-Off Value
$\mathbf{A}, \mathbf{B}$	Image Matrices
$\mathbf{a}_i, \mathbf{b}_i$	Image Blocks
m	Block Dimension
Π_i	FHE Parameter Set

The quantization matrix Q_m can be scaled to prioritize either quality or compression, by multiplying each element of Q_m with k :

$$k = \begin{cases} \frac{100-q}{50} & q > 50 \\ \frac{50}{q} & q < 50 \end{cases}$$

If the quality level q is set to greater than 50, fidelity to the original image is prioritized over compression, whereas values under 50 compress the image more at the cost of losing some fidelity. q and c together define the compression-ratio and the lossiness of the reconstructed image. A lower q also lowers the risk of cutting off non-zero values. The more compression we enforce, the lower the quality will be.

Quantization results in a matrix with a large number of zeros, especially towards the bottom right corner of each matrix. To exploit this redundancy, a *Zigzag* traversal is applied, turning the matrix into a one-dimensional vector, with these zeroes at the end. The function *Inverse_zigzag()* turns a one-dimensional vector back into a matrix, following the same zigzag pattern.

The next step is where we really diverge from the JPEG standard. JPEG uses Huffman encoding [20] to represent this vector. Huffman coding, like other popular encoding schemes, relies on the principle that more frequent symbols should be represented with shorter codes compared to rarer symbols, resulting in an overall compression. This principle is unsuited for FHE, where consistency, constant-runtime, and input-agnosticism are desired above all.

The encoding we suggest (see Algorithm 1) is more simple: we only keep a constant number c of non-zero elements from each block, discarding all other elements. Ideally, this number should be determined independent of the image, such as not to reveal any information about the level of texture present in the data. However, in some applications, c might not leak much information about the image. For example, in simple portrait photos, a busy background can greatly increase the necessary c , while this says nothing about the face in the image. In these cases, the client could pre-compute the ideal c . It should be noted that predetermining this c can result in non-zero values being discarded. This causes additional artifacts (blockiness) in the image, but the effect is usually minimal if the value is well selected. From our experiments, $c = 30$ seemed to be a good choice considering both the SSI and the compression ratio.

Using this encoding, we make sure that every block of the image undergoes the same compression ratio (see also section 3.2). Furthermore, both compression and decompression become very simple actions, and can be efficiently performed under FHE. The encoding itself is lossless, but can be lossy for some highly textured or discrete-tone images. Granted, the data will be compressed less than it would using a more evolved coding scheme (Huffman, Golomb, ...). As discussed earlier, this approach is impractical with homomorphic encryption.

Decompression works very similarly to compression, knowing that in the decoding, we pad with encryptions of 0 until we reach the required size.

Algorithm 1: Encode

Input: blocks $\mathbf{a} = [a_0, a_1, \dots, a_{n-1}]$ with $|a_i| = l_a$
Input: Cut-off point $c < m^2$
Output: blocks $\mathbf{b} = [b_0, b_1, \dots, b_{n-1}]$ with $|b_i| = l_b$ and $l_a \geq l_b$
1 **for** $i \in [0, n-1]$ **do**
2 $b_i \leftarrow a_i[0 : c]$;
3 **end**
4 **return** $\mathbf{b}$

Algorithm 2: Compress

Input: Encrypted image $\mathbf{A}$: split into n $m \times m$ blocks:
 $\mathbf{A} = [A_0, A_1, \dots, A_{n-1}]$ with $A_i \in \mathbb{Z}^{m \times m}$
Input: Cut-off point $c < m^2$
Output: Encrypted, compressed image $\mathbf{B} = [b_0, b_1, \dots, b_{n-1}]$ with $b_i \in \mathbb{Z}^c$
1 **for** $l \in [0, n-1]$ **do**
2 $C_l \leftarrow \text{Level}(A_l)$;
3 $D_l \leftarrow T_m C_l T_m^\top$;
4 **for** $(i, j) \in \mathbb{N}_m^2$ **do**
5 $E_{l,i,j} \leftarrow \lfloor (\frac{D_{l,i,j}}{Q_{m,i,j}}) \rfloor$;
6 **end**
7 $\mathbf{b}_l \leftarrow \text{Zigzag}(E_l)[0..c]$;
8 **end**
9 **return** $\{b_0, b_1, \dots, b_{n-1}\}$

3.2 Non-Constant Compression Ratio

As mentioned before, in regular JPEG, each block gets a different compression rate. This section will show why this is incompatible with FHE.

Consider the photo in figure 2a. The background of the image is blurry, which means that there is less information there. The foreground, the branches and the bird, contain more high-frequency data. In JPEG, these blocks in the foreground would be compressed less than the smoother blocks in the background. Say we implement this directly in FHE. The DCT transforms each block i to the frequency spectrum, representing the block based on the frequencies present in it. A smoother block with less high-frequency information in it would result in a lot of trailing zeros, whereas the DCT of a block with more high-frequency information would not have many zeros at all. In JPEG, the cutoff point c_i is determined by only cutting off zeros. This way, different blocks i get a different c_i . Encryption of these blocks mask the values, but does not hide that there are c_i values representing block i . An adversary listening in, who only knows the c_i , would then still learn some information about the encrypted image: if c_i is relatively large, it means block i contains high-frequency information, and is relatively "busy". If c_i is relatively small, it means block i is more "smooth", containing less high-frequency information, and hence it was possible to compress

Algorithm 3: Decompress

Input: Block size m , which can be equal to either 8 or 16
Input: Encrypted, compressed image $\mathbf{B} = [b_0, b_1, \dots, b_{n-1}]$ with $b_i \in \mathbb{Z}^c$
Output: Encrypted image $\mathbf{A} = [A_0, A_1, \dots, A_{n-1}]$ with $A_i \in \mathbb{Z}^{m \times m}$

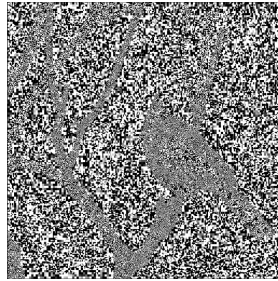
```
1 for  $l \in [0, n-1]$  do
2   for  $i \in [c, m^2]$  do
3      $b_{l,i} \leftarrow \text{Enc}(0)$ 
4   end
5    $C_l \leftarrow \text{Inverse\_Zigzag}(b_l)$  ;
6   for  $(i, j) \in \mathbb{N}_m^2$  do
7      $D_{l,i,j} \leftarrow C_{l,i,j} \cdot Q_{l,i,j}$  ;
8   end
9    $E_l \leftarrow T_m^\top D_l T_m$  ;
10   $A_l \leftarrow \text{Unlevel}(E_l)$  ;
11 end
12 return  $[A_0, A_1, \dots, A_{n-1}]$ 
```

it more. While it may not seem like much information leaking, this is actually enough to make out for example the silhouette of an object or the presence of buildings on a satellite image. For our example bird photo, the leaked information translates to an image such as figure 2b, where we can clearly see the silhouette of the bird and the branches.

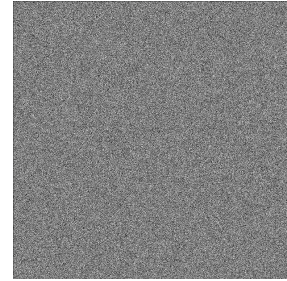
If an adversary were to try this method on our proposed compression scheme with constant block compression, they would see figure 2c, and not learn much.



(a) The original image



(b) Predicted image knowing only the compression rate of each block.



(c) Predicted image when every block is compressed equally.

Fig. 2: Example of the information leak caused by a variable block compression rate.

3.3 Ciphertext Packing

In order to make our algorithms more efficient we exploit the packing capabilities of CKKS [11]. Recall that ciphertext packing refers to the ability to encrypt multiple messages in a single ciphertext.

In our work, we use ciphertext packing as follows. First, the image is divided into blocks of size $m \times m$, after which each block separately undergoes compression. Then, each position in a block is encrypted in a separate ciphertext. We now have c ciphertexts, one for the first position in a block, one for the second, etc. Therefore, if $m = 8$ for example, a decompressed image is represented using 64 ciphertexts, regardless of the dimension of the image. Figure 3 illustrates this principle without compression (for clarity). Furthermore, with this design every block-wise operation becomes trivially parallelizable.

Observe that it is also possible to compress and process multiple images at a time. When compressing k images at once, we obtain the same number of ciphertexts, but they will be k times longer.

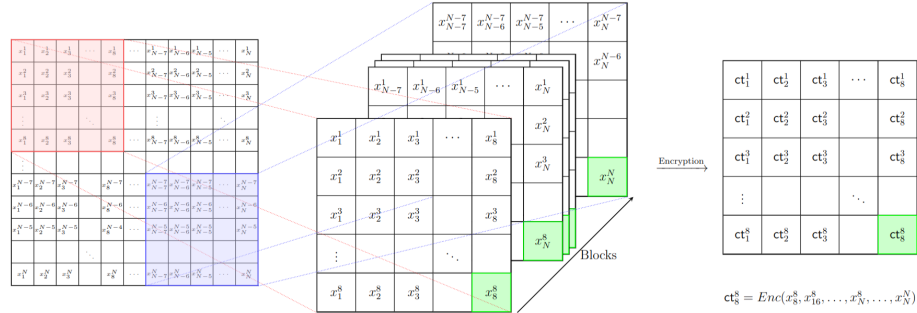


Fig. 3: Illustration of our ciphertext packing

The packing method we opt for, using CKKS, is inspired by the method of [22]. The key difference between [22] and our packing method being the required number of ciphertexts. That is, in [22] one ciphertext is created per location/pixel in an image, needing as many ciphertexts as an image has pixels. In contrast, we create a ciphertext per frequency in a block, which is greatly fewer ciphertexts albeit longer ones. A similar packing technique is presented in CaReNets [10], where images are combined to completely fill the ciphertexts, and leave no slots empty. While this makes the ciphertext more compact, it is not suitable for our situation where the compression and the processing are highly parallelized per block.

3.4 Image Processing

A simple yet powerful and popular method of processing images is to apply convolutional filters on them: We apply filters or kernels on the image, by

convolving the two. These kernels are small matrices, usually of size 3×3 , containing integers or rational numbers. They are constructed for a specific purpose, be it blurring, sharpening, edge detection, etc. A convolution works by passing the kernel over the image and multiplying each value of the kernel with the overlapping value in the image. These products are summed, resulting in the new value for the pixel at the kernel's location.

In Figure 4 we display some examples of kernels for horizontal and vertical edge detection: Prewitt kernels and Sobel kernels. These kernels may be applied for sharpening and blurring as well.

-1	0	1
-2	0	2
-1	0	1

(a) Sobel kernel X

1	2	1
0	0	0
-1	-2	-1

(b) Sobel kernel Y

1	0	-1
1	0	-1
1	0	-1

(c) Prewitt kernel X

-1	-1	-1
0	0	0
1	1	1

(d) Prewitt kernel Y

Fig. 4: Popular Edge Detection Kernels

Our server needs to be able to perform convolutions on the encrypted (but decompressed) blocks of the image. This is hard to parallelize, as applying the convolution requires neighboring pixel values, regardless of block borders. We suggest the following solution: as the very first step, before the client even encrypts/compresses the image, the client can split the image into blocks of size $(m - 2) \times (m - 2)$. Then, these smaller blocks on the edges are padded with the neighboring cells, such that they end up with overlapping blocks of size $m \times m$. This system works for 3×3 kernels, which covers almost all convolutional filters used in practice. Recall that CKKS also provides supports for non-integer kernels.

This not only reduces the compression ratio, but also allows for efficient image processing on the server, and avoids costly server-side packing operations, where the ciphertexts would have to be reconfigured. It is also worth noting that while we strive for a good compression ratio, the goal of this paper is not to achieve the optimal compression ratio. It rather is to find a balance/trade-off between the compression ratio, fidelity and efficiency. We have found that

for this setting, in order to maintain an SSI close to 0.95, most images require $c_8 = 30$ for $m = 8$, leading to a compression ratio of 100 : 83.3. For $m = 16$, most images require $c_{16} = 70$, leading to a compression ratio of 100 : 35.7. The simplest image processing actions, such as inversion and brightening, are pixel-based. Such operations do not require any knowledge of the values of neighboring pixels, and thus do not require our padding trick. In these cases, the client can simply split the image in blocks of size $m \times m$. Here, we find that most images require $c_8 = 22$ (respectively $c_{16} = 63$) to maintain an $SSI \geq 0.95$, leading to a compression ratio of 100 : 34.4 (respectively 100 : 24.6).

In reality, the value of c can be determined for a specific image or a set of images by running the compression and decompression code in the clear, and verifying the image quality. The cut-off point could potentially be quite different after processing compared to before, depending on the operation that was performed.

Alternative Image Processing Techniques. Not all image processing techniques employ convolutional filters. Some operations can be done on individual pixels, without taking into account their neighbors. Think about inverting an image or changing its brightness. Our compression algorithm is compatible with this type of image processing without any modification. We can simply use 8×8 blocks and achieve a greater compression ratio.

Other image processing techniques are based in the frequency-domain: if we take a DCT of the entire image at once, we can increase or decrease the presence of certain frequencies. This is used in blurring or noise-cancellation. Our compression algorithm is not optimal for such techniques, as the server would need to merge the blocks together first, which can be quite costly for big images.

4 Experimental Results

All of our experiments for both client and server were run on a MacBook Pro with an M4 Pro (8P + 4E) CPU and 24 GB of RAM. Our implementation³ has been moderately optimized, but could still be improved. Nonetheless, we think our numbers are promising as it would be safe to assume that the hardware setup we used is less powerful than a standard "server" and more powerful than a standard client.

4.1 Parameters

We ran tests for four different image dimensions, and four algorithms each. The four algorithms are:

1. Pixel-wise processing on 8×8 blocks.

³ <https://github.com/KULeuven-COSIC/img-processing-fhe>

2. Convolutional processing on 8×8 blocks.
3. Pixel-wise processing on 16×16 blocks.
4. Convolutional processing on 16×16 blocks.

Balancing performance, security, and bandwidth is always a challenge with FHE. The larger our parameters, the more we could pack in a ciphertext (Section 3.3) at the cost of larger ciphertext sizes and longer computation times. In an ideal world we would have a parameter set for every single image size and type of operation. However, for practicality, we prioritized fixing the security level at 128 bits for all cases and chose 3 different parameter sets for our experiments.

1. Π_1 for images smaller than or equal to 256×256 .
2. Π_2 for images larger than 256×256 and smaller than or equal to 1024×1024 .
3. Π_3 for images larger than 1024×1024 .

We note that Π_2 is the most fine-tuned among our parameters.

4.2 Timing

Each timing in Table 2 is for one 8-bit grayscale image of mentioned dimensions. The values are averaged over a few randomly selected images: 3 images with dimensions 256×256 , 7 images with 512×512 and 4 images with greater dimensions. The benchmarks are server-side timings, meaning that the compression and decompression are both done homomorphically. Server-side RAM Consumption varied between 1 GB and 13 GB based on the size of the image being processed.

The decompression is more costly than the compression, because the server needs to pad the compressed ciphertext with encryptions of zero and process them as well.

It is clear that the pixel-wise processing is highly efficient, but the 3×3 convolution as well is relatively fast. The largest image size we experimented on is considerably less fast, while the other image sizes are not as different. This has everything to do with the choices of CKKS-parameters. The parameter choices for 1024×1024 images do not allow for secure ciphertexts long enough to fit 2048×2048 images. Remember that the length of each ciphertext is equal to the number of blocks in the image.

We notice that the 8×8 techniques (1 and 2) are consistently faster, and keep the image quality the highest. This is expected, because these quantization matrices are more mature, and because smaller blocks have more efficient matrix multiplication. We do not include example images that have been put through our pipeline, because with SSI-values above 0.9, it is very hard to tell the difference.

When comparing with [27], we highlight that our numbers demonstrate a speedup rate between 173x and 1093x when decompressing the same 256×256 images, depending on the setting. It is also worth noting, however, that while their experiments were performed on a much more powerful machine, they also were run at least 7 years ago. We were unable to retrieve the source-code to run a fair benchmarking using a modern library.

Image Size	Setting	Decompression	Processing	Compression	Total	SSI	Compression Ratio
256×256	1	5.16s	11ms	3.24s	8.41s	0.95	100 : 34.4
256×256	2	6.47s	2.02s	4.58s	13.07s	0.935	100 : 83.3
252×252	3	26.40	68ms	18.72s	45.18s	0.91	100 : 24.6
252×252	4	32.34s	9.48s	29.80s	71.62s	0.905	100 : 35.7
512×512	1	5.17s	11ms	3.23s	8.41s	0.95	100 : 34.4
512×512	2	6.40s	2.00s	4.57s	12.97s	0.935	100 : 83.3
504×504	3	26.22s	69ms	18.74s	45.03s	0.92	100 : 24.6
504×504	4	31.97s	9.57s	30.01s	71.55s	0.92	100 : 35.7
1024×1024	1	4.91s	12ms	3.18s	8.10s	0.98	100 : 34.4
1024×1024	2	6.12s	1.98s	4.53s	12.63s	0.975	100 : 83.3
1022×1022	3	25.68s	73ms	18.57s	44.32s	0.96	100 : 24.6
1022×1022	4	31.96s	9.62s	30.22s	71.80s	0.97	100 : 35.7
2048×2048	1	46.41s	247ms	30.49s	77.14s	0.98	100 : 34.4
2048×2048	2	46.22s	16.31s	35.32s	97.85s	0.98	100 : 83.3
2044×2044	3	67.87s	260ms	44.30s	112.43s	0.975	100 : 24.6
2044×2044	4	67.45s	19.68s	62.08s	149.21s	0.975	100 : 35.7

Table 2: Experimental Timing Results (Server-Side)

4.3 Bandwidth

In contrast to running time, the bandwidth required to send an image from client to server can be computed as a function of our selected parameters Π from Section 4.1 and our cut-off point c from Algorithm 2.

Moreover, since the server’s ciphertexts that are intended to be sent back to the client for decryption contain less information by design (ref. Section 3.3), we can benefit from a reduced size on responses.

Table 3 demonstrates our bandwidth consumption, highlighting that Π_2 is our most versatile parameter set. We remind that tailor-made parameters per requirement, configuration and image size will always bring forth an improvement on the numbers below.

4.4 Comparison with the Current State of the Art

Zama’s HuggingFace demo [33] is the only other public proof-of-concept implementation that works end-to-end. A comparison between our implementation and theirs is not a direct "apples-to-apples" comparison because of the following points:

- They only support up images of up to 100×100 pixels in size.
- The demo is built using TFHE, a different FHE scheme that has pros and cons compared to CKKS.
- Their experiments are run with RGB images whereas we ran our experiments with grayscale.

Parameter Set	$c_8 = 22$ (Setting 1)	$c_8 = 30$ (Setting 2)	$c_{16} = 63$ (Setting 3)	$c_{16} = 70$ (Setting 4)
Π_1	$\rightarrow 90.2$ MB $\leftarrow 24.2$ MB	$\rightarrow 123$ MB $\leftarrow 33$ MB	$\rightarrow 258$ MB $\leftarrow 69.3$ MB	$\rightarrow 287$ MB $\leftarrow 77$ MB
Π_2	$\rightarrow 123.2$ MB $\leftarrow 24.2$ MB	$\rightarrow 168$ MB $\leftarrow 33$ MB	$\rightarrow 352.8$ MB $\leftarrow 69.3$ MB	$\rightarrow 392$ MB $\leftarrow 77$ MB
Π_3	$\rightarrow 1034$ MB $\leftarrow 134.2$ MB	$\rightarrow 1410$ MB $\leftarrow 183$ MB	$\rightarrow 2961$ MB $\leftarrow 384.3$ MB	$\rightarrow 3290$ MB $\leftarrow 427$ MB

Table 3: Experimental bandwidth consumption for client requests ($\rightarrow$) and server responses ($\leftarrow$) given different parameter sets and settings

- They do not compress or encode images: a raw pixel-matrix is encrypted and sent to the server, therefore messages between client and server are equal in length and overall server running time will be lower in comparison to ours.
- Supported Convolution Operations (corr. our Setting 1): Blurring, Ridge Detection, Sharpening.
- Supported Pixel-Wise Operations (corr. our Setting 2): Identity, Rotation, Inversion.

In order to level out the comparison, we can assume that a 100×100 RGB image has an equivalent required bandwidth to a 300×300 grayscale image. We also cloned their code and ran it on the same machine we used for our experiments above. The results can be found in Table 4.

Operation Type	Running Time	Bandwidth Consumed
Pixel-Wise	360ms	$\rightarrow 177$ MB $\leftarrow 177$ MB
Convolution	650ms	$\rightarrow 235$ MB $\leftarrow 235$ MB

Table 4: Performance of Zama’s TFHE-based Image Processing Demo for Colored 100×100 Images

Interpretation When comparing Zama’s running time with ours, they clearly stand at an overall advantage since no compression/decompression is taking place. If we perform a less rigorous comparison between their running time and only our processing time from the first two rows of Table 2, we can estimate that the numbers are comparable for performing convolutions while we seem to be at a disadvantage when it comes to pixel-wise processing.

However, the more interesting result emerges when we compare bandwidth: for us to transmit a single channel 1024×1024 image back and forth for convolutional processing, we require 147.4 MB whereas they require 470 MB for a 100×100 RGB image. This is approximately 3x more bandwidth for $\sim 30\%$ of the amount of pixels.

If Zama’s TFHE solution scales to support higher-resolutions, it might eventually be practical in scenarios where only a predefined set of image-processing operations is interesting and the bandwidth cost between clients and server is negligible. Our solution for the time being natively supports generic operations, works for much higher resolutions, at a lower bandwidth-cost but higher-computational overhead which can be offset by a highly performing server, and more refined parameters.

5 Summary and Future Directions

In this work, we described a novel technique to homomorphically compress, process and decompress natural images. This is a fundamental step towards improved and more advanced privacy-preserving image manipulation, which is paramount in many important use-cases in computer vision such as biometric-based authentication and computer-aided medical diagnostics.

Prior to our work, the literature focused mainly on private image processing, leaving compression and decompression aside. We believe that providing the capability to compress and decompress images privately opens the door to the construction of more efficient, easy-to-use frameworks while reducing the bandwidth and memory requirements of the applications.

We discussed the two types of packing we use, depending on the kind of processing that should be applied (3×3 convolution or pixel-wise processing). Our implementation shows that the techniques using 8×8 blocks are more time-efficient, while the 16×16 blocks achieve more compression.

The next natural step would be to generalize our compression algorithm to make it compatible with more complex image processing, such as facial recognition and object detection.

Interesting orthogonal work could combine our results with the recent advancements on Zero Knowledge Proofs to support proofs of image provenance [5, 14] or to certify the server’s processing on the image.

Finally, an interesting question is whether it is possible to conclusively prove that entropy encoding is impractical in combination with FHE. If not, it should be possible to further improve the compression ratio we achieve.

Acknowledgements

This work was supported by the Flemish Government through the Cybersecurity Research Program with grant number: VOEWICS02 and by the FWO under an Odysseus project GOH9718N. This work was supported by the Spanish Ministry of Economy and Competitiveness with reference number RTI2018-102112-B-I00. Any opinions, findings and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of Cybersecurity Research Flanders, the FWO or the Spanish Ministry of Economy and Competitiveness.

References

1. Alexandru, A., Kim, A., Polyakov, Y.: General functional bootstrapping using CKKS. Cryptology ePrint Archive, Paper 2024/1623 (2024), <https://eprint.iacr.org/2024/1623>
2. Almohammad, A., Ghinea, G., Hierons, R.M.: Jpeg steganography: A performance evaluation of quantization tables. In: 2009 International Conference on Advanced Information Networking and Applications. pp. 471–478 (2009). <https://doi.org/10.1109/AINA.2009.67>
3. Balla, S., Koushanfar, F.: Heliks: HE linear algebra kernels for secure inference. In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (eds.) Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26–30, 2023. pp. 2306–2320. ACM (2023). <https://doi.org/10.1145/3576915.3623136>, <https://doi.org/10.1145/3576915.3623136>
4. Bergerat, L., Chillotti, I., Ligier, D., Orfila, J.B., Tap, S.: TFHE gets real: an efficient and flexible homomorphic floating-point arithmetic. Cryptology ePrint Archive, Paper 2025/257 (2025), <https://eprint.iacr.org/2025/257>
5. Boneh, D., Grotto, A.J., McDaniel, P., Papernot, N.: Preparing for the age of deepfakes and disinformation. HAI Policy Brief (2020)
6. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical GapSVP. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. LNCS, vol. 7417, pp. 868–886. Springer, Heidelberg (Aug 2012). https://doi.org/10.1007/978-3-642-32009-5_50
7. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) fully homomorphic encryption without bootstrapping. In: Goldwasser, S. (ed.) ITCS 2012. pp. 309–325. ACM (Jan 2012). <https://doi.org/10.1145/2090236.2090262>
8. Brakerski, Z., Vaikuntanathan, V.: Lattice-based FHE as secure as PKE. In: Naor, M. (ed.) ITCS 2014. pp. 1–12. ACM (Jan 2014). <https://doi.org/10.1145/2554797.2554799>
9. Canard, S., Carpov, S., Kuate, D.N., Sirdey, R.: Running compression algorithms in the encrypted domain: a case-study on the homomorphic execution of RLE. Cryptology ePrint Archive, Report 2017/392 (2017), <https://eprint.iacr.org/2017/392>
10. Chao, J., Badawi, A.A., Unnikrishnan, B., Lin, J., Mun, C.F., Brown, J.M., Campbell, J.P., Chiang, M.F., Kalpathy-Cramer, J., Chandrasekhar, V.R., Krishnaswamy, P., Aung, K.M.M.: Carenets: Compact and resource-efficient CNN

- for homomorphic inference on encrypted medical images. CoRR **abs/1901.10074** (2019), <http://arxiv.org/abs/1901.10074>
11. Cheon, J.H., Kim, A., Kim, M., Song, Y.S.: Homomorphic encryption for arithmetic of approximate numbers. In: Takagi, T., Peyrin, T. (eds.) ASIACRYPT 2017, Part I. LNCS, vol. 10624, pp. 409–437. Springer, Heidelberg (Dec 2017). https://doi.org/10.1007/978-3-319-70694-8_15
 12. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In: Cheon, J.H., Takagi, T. (eds.) ASIACRYPT 2016, Part I. LNCS, vol. 10031, pp. 3–33. Springer, Heidelberg (Dec 2016). https://doi.org/10.1007/978-3-662-53887-6_1
 13. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology* **33**(1), 34–91 (Jan 2020). <https://doi.org/10.1007/s00145-019-09319-x>
 14. Datta, T., Boneh, D.: Using zk proofs to fight disinformation. <https://medium.com/@boneh/using-zk-proofs-to-fight-disinformation-17e7d57fe52f> (2022)
 15. Fan, J., Vercauteren, F.: Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive, Report 2012/144* (2012), <https://eprint.iacr.org/2012/144>
 16. Fleischhacker, N., Larsen, K.G., Simkin, M.: Compressing encrypted data over small fields. *Cryptology ePrint Archive, Paper 2023/946* (2023), <https://eprint.iacr.org/2023/946>, <https://eprint.iacr.org/2023/946>
 17. Fu, W., Lin, R., Inge, D.: Fully homomorphic image processing. CoRR **abs/1810.03249** (2018), <http://arxiv.org/abs/1810.03249>
 18. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: Mitzenmacher, M. (ed.) 41st ACM STOC. pp. 169–178. ACM Press (May / Jun 2009). <https://doi.org/10.1145/1536414.1536440>
 19. Golomb, S.: Run-length encodings (corresp.). *IEEE Transactions on Information Theory* **12**(3), 399–401 (1966). <https://doi.org/10.1109/TIT.1966.1053907>
 20. Huffman, D.A.: A method for the construction of minimum-redundancy codes. *Proceedings of the IRE* **40**(9), 1098–1101 (1952). <https://doi.org/10.1109/JRPROC.1952.273898>
 21. International Organisation for Standardisation: Information technology – Digital compression and coding of continuous-tone still images - Requirements and guidelines. Standard, CCITT, Geneva, CH (Sep 1992)
 22. Jain, N., Nandakumar, K., Ratha, N.K., Pankanti, S., Kumar, U.: Optimizing homomorphic encryption based secure image analytics. In: 23rd International Workshop on Multimedia Signal Processing, MMSP 2021, Tampere, Finland, October 6–8, 2021. pp. 1–6. IEEE (2021). <https://doi.org/10.1109/MMSP53017.2021.9733620>, <https://doi.org/10.1109/MMSP53017.2021.9733620>
 23. Jiang, X., Kim, M., Lauter, K.E., Song, Y.: Secure outsourced matrix computation and application to neural networks. In: Lie, D., Mannan, M., Backes, M., Wang, X. (eds.) *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15–19, 2018*. pp. 1209–1222. ACM (2018). <https://doi.org/10.1145/3243734.3243837>, <https://doi.org/10.1145/3243734.3243837>
 24. Kelsey, J.: Compression and information leakage of plaintext. In: Daemen, J., Rijmen, V. (eds.) *FSE 2002. LNCS, vol. 2365*, pp. 263–276. Springer, Heidelberg (Feb 2002). https://doi.org/10.1007/3-540-45661-9_21
 25. Kuate, D.N., Canard, S., Sirdey, R.: Towards video compression in the encrypted domain: A case-study on the H264 and HEVC macroblock processing pipeline.

- In: Camenisch, J., Papadimitratos, P. (eds.) *Cryptology and Network Security - 17th International Conference, CANS 2018, Naples, Italy, September 30 - October 3, 2018, Proceedings. Lecture Notes in Computer Science*, vol. 11124, pp. 109–129. Springer (2018). https://doi.org/10.1007/978-3-030-00434-7_6, https://doi.org/10.1007/978-3-030-00434-7_6
26. Lee, E., Lee, J., Lee, J., Kim, Y., Kim, Y., No, J., Choi, W.: Low-complexity deep convolutional neural networks on fully homomorphic encryption using multiplexed parallel convolutions. In: Chaudhuri, K., Jegelka, S., Song, L., Szepesvári, C., Niu, G., Sabato, S. (eds.) *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA. Proceedings of Machine Learning Research*, vol. 162, pp. 12403–12422. PMLR (2022), <https://proceedings.mlr.press/v162/lee22e.html>
 27. Ma, X., Liu, C., Cao, S., Zhu, B.: JPEG decompression in the homomorphic encryption domain. In: Boll, S., Lee, K.M., Luo, J., Zhu, W., Byun, H., Chen, C.W., Lienhart, R., Mei, T. (eds.) *2018 ACM Multimedia Conference on Multimedia Conference, MM 2018, Seoul, Republic of Korea, October 22-26, 2018*. pp. 905–913. ACM (2018). <https://doi.org/10.1145/3240508.3240672>, <https://doi.org/10.1145/3240508.3240672>
 28. P, N., Sathya, M., Vengattaraman, T.: Survey on image compression algorithms and their efficiencies under different types of files. In: *2023 6th International Conference on Recent Trends in Advance Computing (ICRTAC)*. pp. 238–246 (2023). <https://doi.org/10.1109/ICRTAC59277.2023.10480806>
 29. Pennebaker, W.B., Mitchell, J.L.: *JPEG still image data compression standard*. Van Nostrand Reinhold, New York (N.Y.) (1993)
 30. Rovida, L., Leporati, A.: Encrypted image classification with low memory footprint using fully homomorphic encryption. *Int. J. Neural Syst.* **34**(5), 2450025:1–2450025:16 (2024). <https://doi.org/10.1142/S0129065724500254>, <https://doi.org/10.1142/S0129065724500254>
 31. Vengadapurvaja, A., Nisha, G., Aarth, R., Sasikaladevi, N.: An efficient homomorphic medical image encryption algorithm for cloud storage security. *Procedia Computer Science* **115**, 643–650 (2017). <https://doi.org/https://doi.org/10.1016/j.procs.2017.09.150>, <https://www.sciencedirect.com/science/article/pii/S1877050917319798>, 7th International Conference on Advances in Computing & Communications, ICACC-2017, 22-24 August 2017, Cochin, India
 32. Wang, Z., Simoncelli, E., Bovik, A.: Multiscale structural similarity for image quality assessment. In: *The Thrity-Seventh Asilomar Conference on Signals, Systems & Computers*, 2003. vol. 2, pp. 1398–1402 Vol.2 (2003). <https://doi.org/10.1109/ACSSC.2003.1292216>
 33. Zama: Image filtering on encrypted data using fully homomorphic encryption. https://huggingface.co/spaces/zama-fhe/encrypted_image_filtering (2023)
 34. Zheng, P., Huang, J.: An efficient image homomorphic encryption scheme with small ciphertext expansion. In: Jaimes, A., Sebe, N., Boujemaa, N., Gatica-Perez, D., Shamma, D.A., Worring, M., Zimmermann, R. (eds.) *ACM Multimedia Conference, MM '13, Barcelona, Spain, October 21-25, 2013*. pp. 803–812. ACM (2013). <https://doi.org/10.1145/2502081.2502105>, <https://doi.org/10.1145/2502081.2502105>
 35. Zinner, T., Hohlfeld, O., Abboud, O., Hossfeld, T.: Impact of frame rate and resolution on objective qoe metrics. In: *2010 Second International Workshop on*

Quality of Multimedia Experience (QoMEX). pp. 29–34 (2010). <https://doi.org/10.1109/QoMEX.2010.5518277>